VERGE

APPLICATION SECURITY ASSESSMENT REPORT

AUTHENTICATION, SESSION AND IDENTITY SUBSYSTEMS

CLIENT: OWASP Foundation
RELEASED: 09 November, 2026

alex.vergesec@gmail.com

CLASSIFIED AS CONFIDENTIAL

TABLE OF CONTENTS

APPLICATION SECURITY ASSESSMENT REPORT 3
SECTION 1: EXECUTIVE SUMMARY & ENGAGEMENT OVERVIEW 4
1.1 Executive Risk Narrative 4
1.2 Engagement Scope Boundaries 4
1.3 Consolidated Findings Inventory (Vulnerabilities) 5
1.4 Verified Security Controls Inventory (PASS Observations) 5
SECTION 2: DETAILED VULNERABILITY FINDINGS 6
Finding 2.1: Account Takeover via Automated Security Question Brute-Force 6
Finding 2.2: Permanent Account Takeover via Chained DOM-Based XSS & Unthrottled Password Modification 9
Finding 2.3: Insecure State-Changing Operations on Profile Management Endpoints via Cross-Site Request Forgery (CSRF) 13
Finding 2.4: Primary Login Vulnerability to unrestricted Credential Stuffing 16
SECTION 3: VERIFIED DEFENSIVE CONTROLS (PASS OBSERVATIONS) 18
CTRL-01: Input Sanitization & Output Encoding on User Profile Subsystem 18
CTRL-02: Array Input Rejection & Parameter Pollution Resistance on Login Payload 18
CTRL-03: Response Timing Uniformity & Side-Channel Resistance on Login 18
CTRL-04: Generic Error Messaging & Response Uniformity on Primary Authentication 19
CTRL-05: Synchronous Atomic Processing & Parallel Rate-Limiting Resistance on Account Recovery 19
CTRL-06: Pitchfork Attack Counter Persistence on Account Recovery 19
CTRL-07: HTTP Method Override Directives Rejection on State Changing Routes 19
SECTION 4: APPENDICES & FORMAL SIGN-OFF 20
Appendix A: CVSS v3.1 Severity Rating Framework 20
Appendix B: Developer Remediation Action Plan Checklist 21
Appendix C: Assessment Environment & Testing Tooling 21
Appendix D: Artifact Cleanup Attestation & Formal Sign-Off 22

APPLICATION SECURITY ASSESSMENT REPORT

Web Application Penetration Testing & Security Architecture Review

PROPERTY	SPECIFICATION DETAIL
TARGET APPLICATION	OWASP Juice Shop
APPLICATION VERSION	v20.2.0 (Local Deployment)
TARGET ARCHITECTURE	REST API & Node.js / Angular Web Subsystem
TESTING SCOPE	Cross-Site Scripting (XSS), Authentication, Session Management, Profile Updates, Password Reset
TESTING METHODOLOGY	Black-Box / Grey-Box Web Application Penetration Testing
ACTIVE TESTING WINDOW	September 16, 2026 – September 25, 2026
REPORT COMPLETION DATE	September 27, 2026
ASSESSOR ENTITY	Verge Security Labs

1.0 Document Administration & Control

1.1 Version Control

AUTHOR	DELIVERY DATE	PAGES	VERSION	STATUS
Alex Vance	November 9, 2026	22	v1.0	Final Deliverable

1.2 Document Distribution

NAME	TITLE	ORGANIZATION
Engineering Leadership Team	Chief Technology Officer	OWASP Foundation

SECTION 1: EXECUTIVE SUMMARY & ENGAGEMENT OVERVIEW

1.1 Executive Risk Narrative

During the active testing window from September 16, 2026 to September 19, 2026, an offensive security evaluation was conducted against the OWASP Juice Shop web application (v20.2.0 local deployment). The assessment focused on evaluating the resilience of critical cross-site scripting (XSS), authentication, session management, user profile, and password recovery subsystems against real-world threat actors under a combined Black-Box and Grey-Box methodology.

Testing identified **4 security vulnerabilities (3 Critical, 1 Medium)**, including unauthenticated account takeover via security question brute-forcing (POST /rest/user/reset-password), client-side DOM XSS chained with unthrottled re-authentication (GET /rest/products/search & GET /rest/user/change-password), primary credential stuffing (POST /rest/user/login), and cross-site request forgery across profile endpoints (POST /profile & POST /profile/image/url).

Conversely, technical testing verified **7 resilient defensive baseline controls (PASS observations)**, demonstrating effective input sanitization on profile settings, array input handling on login, constant-time hash execution, synchronous atomic recovery processing, and strict method override rejection.

Real-World Business Risks

- **Account Takeover via Security Question Brute-Force:** Unauthenticated attackers can enumerate valid email accounts via pre-filled security prompts, leverage response body length differentials (829 bytes vs 503 bytes), and bypass weak rate limits (100 requests) to reset user credentials.
- **Permanent ATO via Chained DOM XSS & Unthrottled Password Modification:** Attackers can hijack active sessions via client-side DOM XSS in search results (<img src=1 onerror=...>), exfiltrate tokens from unflagged cookies (missing HttpOnly), and execute unthrottled current-password brute-forcing on /rest/user/change-password to permanently lock out legitimate users.
- **Automated Credential Stuffing on Primary Login Endpoint:** The primary authentication handler (POST /rest/user/login) permits unlimited brute-force dictionary attacks against user credentials due to missing IP throttling and anti-automation controls.
- **Insecure State-Changing Operations via Cross-Site Request Forgery (CSRF):** Profile update endpoints (POST /profile and POST /profile/image/url) lack server-side anti-CSRF token validation and explicit SameSite cookie flags, allowing cross-site state manipulation under specific browser execution windows.

1.2 Engagement Scope Boundaries

TARGET ENDPOINT / URL	ENVIRONMENT	TESTING SCOPE
POST /rest/user/login & /rest/user/reset-password	Staging / Local v20.2.0	Primary Authentication & Password Recovery Audit (In-Scope)
GET /rest/products/search & /rest/user/change-password	Staging / Local v20.2.0	Client-Side DOM XSS & Re-Authentication Chaining (In-Scope)
POST /profile & /profile/image/url	Staging / Local v20.2.0	Profile Management & Anti-CSRF Resilience Audit (In-Scope)

1.3 Consolidated Findings Inventory (Vulnerabilities)

POINT	TITLE	CWE-ID	CWE CATEGORY	SEVERITY
FIND-01	Account Takeover via Automated Security Question Brute-Force	CWE-640	A07:2025 – Identification and Authentication Failures	**CRITICAL**
FIND-02	Permanent ATO via Chained DOM XSS & Unthrottled Password Change	CWE-79 / CWE-307	A03:2025 – Injection & A07:2025 – Identification and Authentication Failures	**CRITICAL**
FIND-03	Insecure State-Changing Operations on Profile Management Endpoints via Cross-Site Request Forgery (CSRF)	CWE-352	A01:2025 – Broken Access Control	**MEDIUM**
FIND-04	Primary Login Vulnerability to Unrestricted Credential Stuffing	CWE-307	A07:2025 – Identification and Authentication Failures	**CRITICAL**

1.4 Verified Security Controls Inventory (PASS Observations)

POINT	SECURITY FEATURE VERIFIED	EVALUATED ENDPOINT	STANDARD MAPPING	STATUS
CTRL-01	Generic Error Messaging & Anti-Username Enumeration	POST /rest/user/login	A07:2025 – Identification and Authentication Failures	**PASS**
CTRL-02	Array Input Handling	POST /rest/user/login	A05:2025 – Security Misconfiguration	**PASS**
CTRL-03	Response Timing Uniformity & Constant-Time Hash Execution	POST /rest/user/login	A02:2025 – Cryptographic Failures	**PASS**
CTRL-04	Input Sanitization & HTML Output Encoding	POST /profile	A03:2025 – Injection	**PASS**
CTRL-05	Synchronous Atomic Processing & Race Condition Resistance	POST /rest/user/reset-password	A04:2025 – Insecure Design	**PASS**
CTRL-06	Pitchfork Counter Persistence across Interleaved Requests	POST /rest/user/reset-password	A07:2025 – Identification and Authentication Failures	**PASS**
CTRL-07	Strict Method Override Directives Rejection	POST /profile & /change-username	A01:2025 – Broken Access Control	**PASS**

SECTION 2: DETAILED VULNERABILITY FINDINGS

Finding 2.1: Account Takeover via Automated Security Question Brute-Force

SEVERITY: CRITICAL	CWE-ID: CWE-640	CVSS SCORE: 8.1
AFFECTED POINT	POST /rest/user/reset-password (answer JSON field)	
OWASP TOP 10	A07:2025 – Identification and Authentication Failures	

Vulnerability Description & Technical Mechanics

The application's account recovery process lets unauthenticated users reset passwords by answering security questions. While the web interface displays generic error text, two technical flaws allow attackers to execute full account takeover:

Client-Side Enumeration & Question Prompt Leak: Entering a registered email into the recovery form automatically fetches and pre-fills the user's saved security question prompt (e.g., "Your eldest sibling's middle name?"). Entering an unregistered email fails to load a prompt. Although this phase requires manual entry, it confirms valid accounts and reveals the exact question category, allowing an attacker to build a focused dictionary list.

Backend Response Metadata Discrepancy: When submitting password reset attempts to /rest/user/reset-password, the backend returns a 401 Unauthorized response with a body size of 503 bytes for incorrect answers, but a 200 OK response with a body size of 829 bytes for correct answers. Automated tools (like Burp Intruder) easily filter by status code (200) or response size (829) to pinpoint successful resets.

Weak Rate-Limiting Threshold: The endpoint permits up to 100 requests per window (X-RateLimit-Limit: 100), providing excessive headroom for brute-forcing low-entropy security answers.

Steps to Reproduce (Proof of Concept)

1. Target Selection: Open http://localhost:3000/#/forgot-password and enter a target email address (e.g., Jordan.Sterling@example.org).

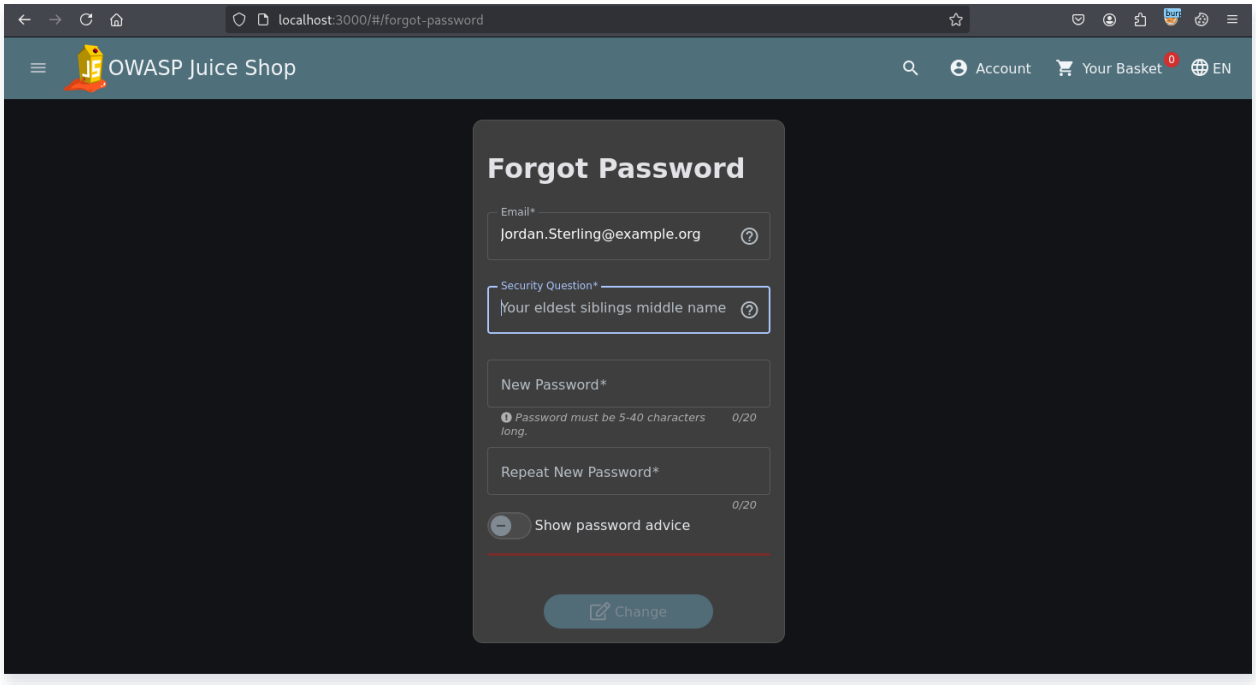


Figure 2.1A: Entering a registered email automatically populates the user's specific security question, confirming account existence and leaking the question domain.

2. Intercept & Configure Intruder: Intercept the password reset submission sent to /rest/user/reset-password using Burp Suite Proxy. Send the request to Burp Intruder, select “Sniper attack”, mark the answer parameter in the JSON payload, and load a targeted wordlist of common middle names.

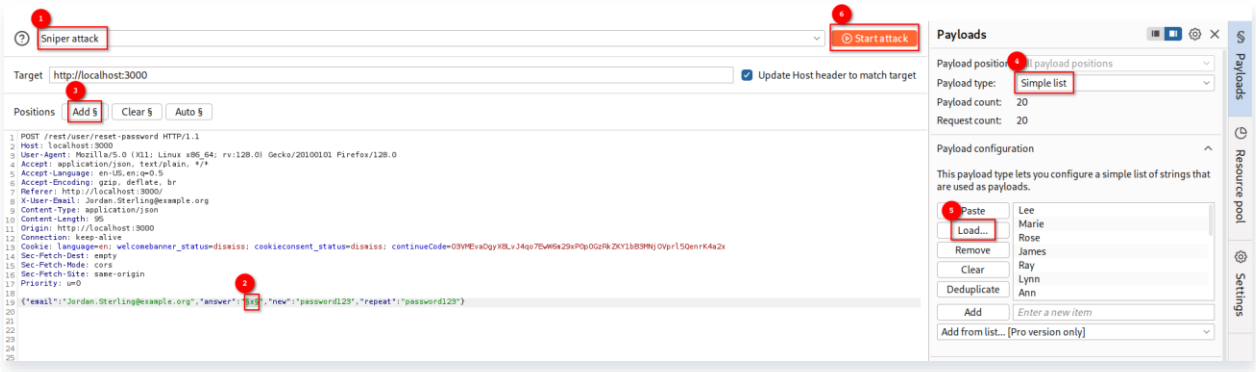


Figure 2.1B: Configuring Burp Intruder to brute-force the `answer` parameter with a targeted list of common middle names.

3. Execute Attack & Verify Success: Execute the attack. Observe that incorrect guesses return 401 Unauthorized (length 503), whereas the correct guess returns 200 OK (length 829). The response headers confirm a permissive threshold of X-RateLimit-Limit: 100, allowing the brute-force attack to succeed well within the allowed quota.

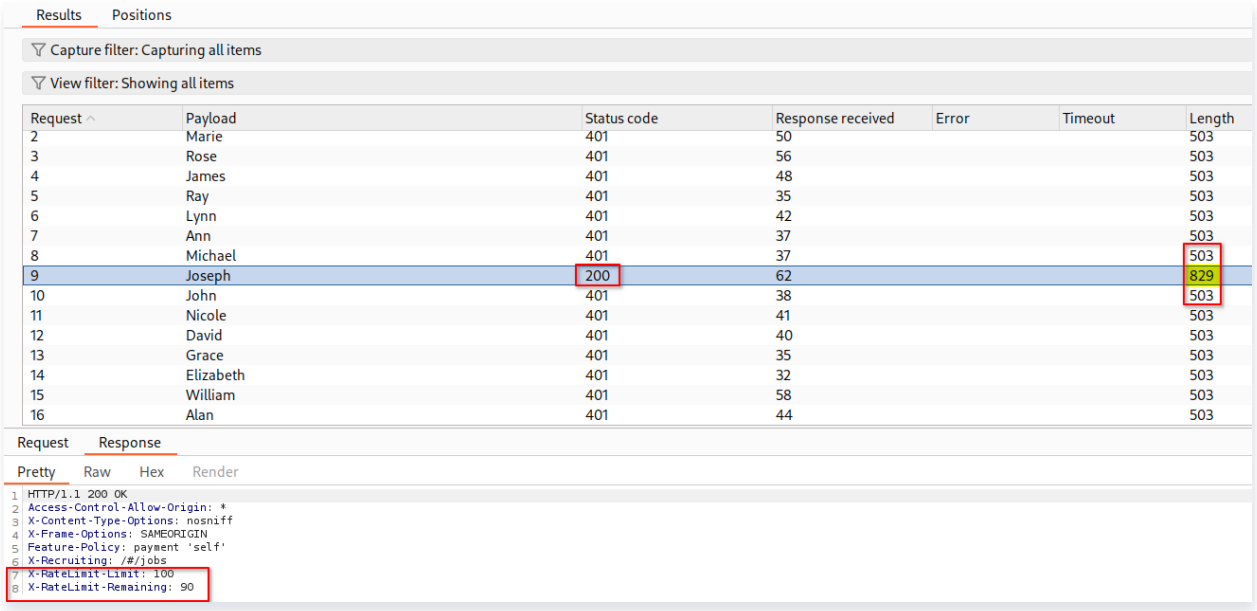


Figure 2.1C: Intruder attack results highlighting the HTTP 200 success response (length 829 vs 503) and permissive rate-limit headers (`X-RateLimit-Limit: 100`).

Risk & Business Impact

An unauthenticated attacker can systematically enumerate user accounts and force password resets without requiring access to secondary authentication factors. Because security questions rely on low-entropy answers, a rate limit of 100 requests almost guarantees account compromise.

Remediation Guidance

1. Migrate to Email-Based Out-of-Band Recovery & Disable UI Pre-filling: Remove frontend functionality that automatically queries backend routes to pre-populate security questions upon email entry. Re-architect the password recovery workflow to require out-of-band email verification (e.g., dispatching a single-use reset link or short-lived token) prior to permitting credentials modifications.

2. Implement Uniform HTTP Redirection & Generic DOM Error Rendering: Configure recovery handlers (/rest/user/reset-password) to issue a uniform HTTP 302 Redirect back to the recovery page upon form submission, regardless of input validity. Upon redirection, the client-side DOM should render a single generic notification via frontend scripts: "If an account exists for this email address, password reset instructions have been sent." Returning identical HTTP status codes and wire-level response lengths neutralizes response oracle analysis (such as Burp Intruder).

3. Enforce Rate-Limiting & CAPTCHA Challenges: Reduce the rate-limit ceiling on /rest/user/reset-password from 100 requests down to 3–5 attempts per hour per IP/account. Additionally, deploy a CAPTCHA challenge (e.g., reCAPTCHA v3 or Cloudflare Turnstile) after 2 consecutive failed attempts to block automated brute-forcing tools before rate limits are reached.

Finding 2.2: Permanent Account Takeover via Chained DOM-Based XSS & Unthrottled Password Modification

SEVERITY: CRITICAL	CWE-ID: CWE-79 / CWE-307	CVSS SCORE: 8.8
AFFECTED POINT	GET /rest/products/search (q parameter) & GET /rest/user/change-password (current parameter)	
OWASP TOP 10	A03:2025 – Injection & A07:2025 – Identification and Authentication Failures	

Vulnerability Description & Technical Mechanics

The application contains an exploitable vulnerability chain combining a client-side DOM-based Cross-Site Scripting (DOM XSS) vector with an unthrottled re-authentication handler. By luring an authenticated victim to a specially crafted search link, an adversary can bypass client-side DOM parsing constraints to extract active session tokens and rapidly brute-force the user's current password, culminating in permanent Account Takeover (ATO).

Client-Side DOM Sink & HTML5 Parsing Constraint Bypass (GET /rest/products/search): The search handler dynamically reflects the q URL query parameter into a client-side innerHTML sink without sanitization or HTML entity encoding. While standard <script> tag injection attempts fail—because the HTML5 specification marks scripts dynamically inserted via innerHTML as already executed—an adversary can bypass this constraint using an HTML element event-handler attribute vector (such as <img src=1 onerror=...>). The browser's HTML parser causes the inline onerror callback to fire during element instantiation inside the DOM, successfully executing arbitrary JavaScript within the victim's authenticated session context.

Session Token Extraction (document.cookie): Because authentication tokens lack the HttpOnly cookie attribute, script execution inside the application origin grants full access to the document.cookie DOM object. Triggering alert(document.cookie) visually proves both successful script execution and unencrypted session cookie retrieval.

Unthrottled Re-Authentication Brute-Force (GET /rest/user/change-password): To finalize account takeover via /rest/user/change-password, the adversary must supply the victim's current password. The backend handler lacks rate-limiting, account lockout mechanisms (only for the attacker), or CAPTCHA validation.

Steps to Reproduce (Proof of Concept)

1. Construct & Navigate to Payload Link: Craft an HTML image element payload utilizing an inline onerror event handler to execute alert(document.cookie) and bypass HTML5 innerHTML script execution rules. Navigate to the payload URL within an authenticated victim session: http://localhost:3000/#/search?q=<img src=x onerror=alert(document.cookie)>

2. Verify DOM Execution & Unencrypted Session Token Access: Upon page render, the browser assigns the query parameter (q) directly to the innerHTML sink, firing the onerror callback and displaying an alert dialog with the victim's active session token (token= eyJ0eXAiOiJKV1Q...).

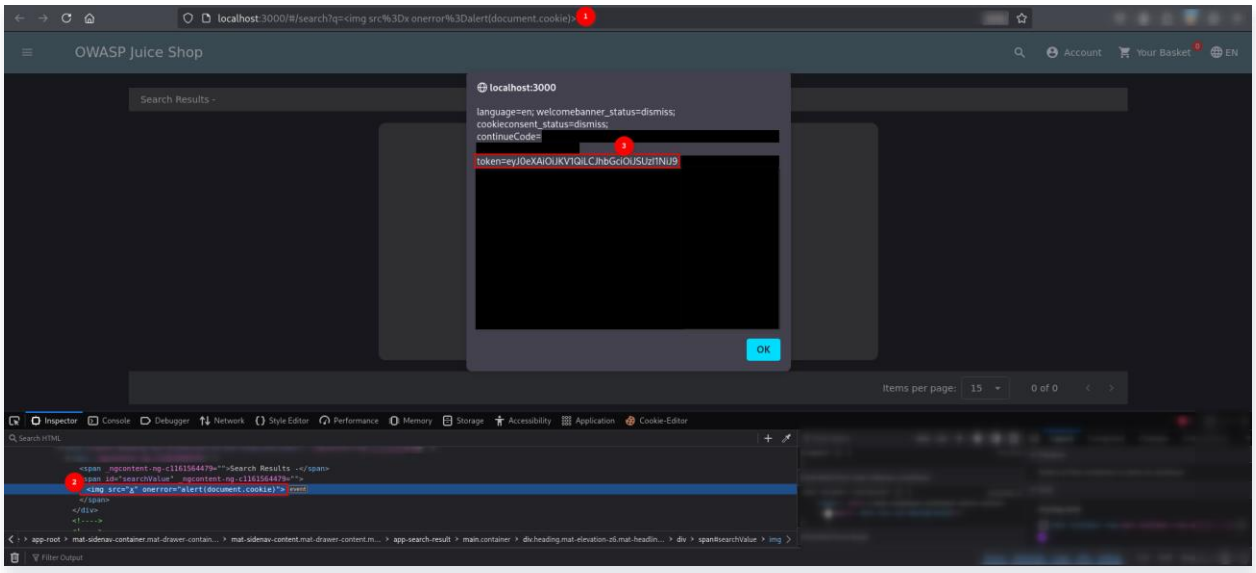


Figure 2.2A: DOM XSS execution triggering alert(document.cookie) dialog box displaying active session token.

3. Configure Automated Current Password Fuzzing in Burp Intruder: Authenticate as the target victim, intercept the password modification request targeting GET /rest/user/change-password?current=...&new=...&repeat=..., and transfer it to Burp Intruder. Set a **Sniper attack** position on the current parameter, load a password dictionary, and initiate the attack.

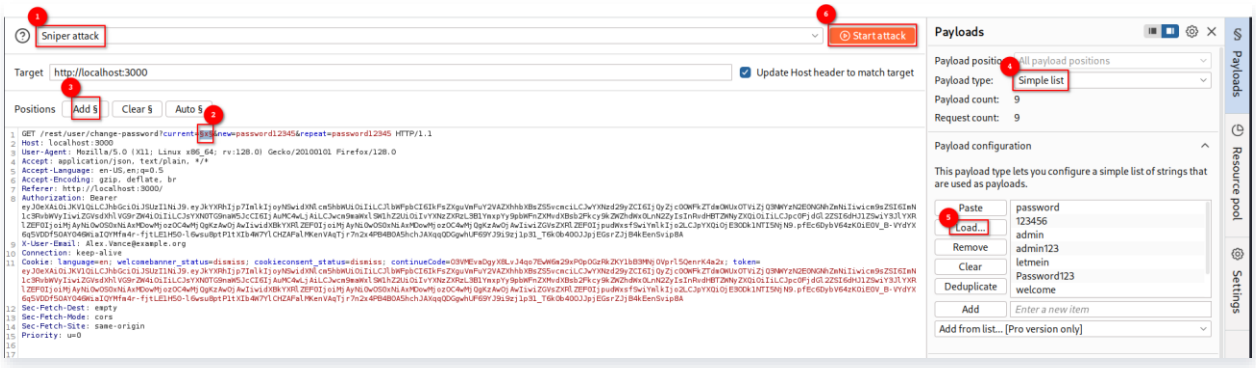


Figure 2.2B: Configuring Burp Intruder Sniper attack position on the current (password) parameter

4. Confirm Permanent Account Takeover (ATO): Analyze the Intruder results table. Observe that invalid password attempts return HTTP 401 Unauthorized (419 bytes), whereas the valid credential (Password123) yields HTTP 200 OK (742 bytes), confirming successful current password discovery and permanent credential overwrite.

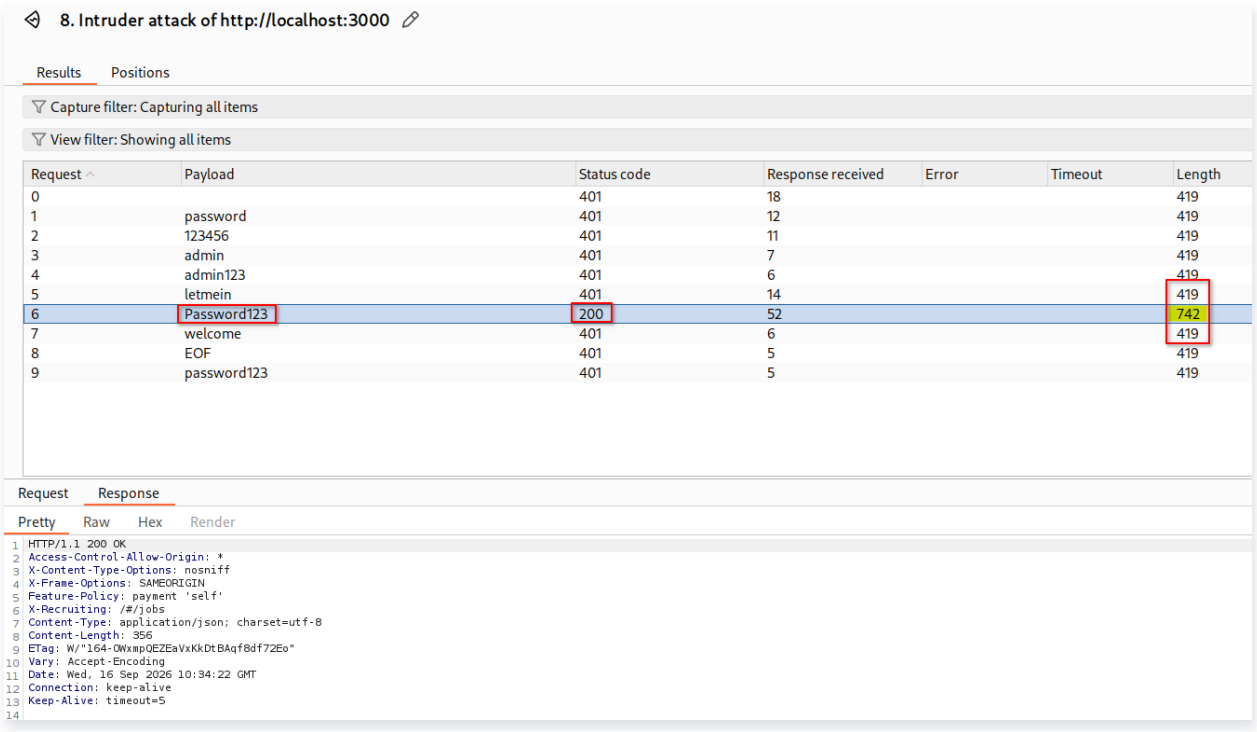


Figure 2.2C: Burp Intruder attack results highlighting response length variance (742 bytes vs 419 bytes) confirming valid password discovery.

Risk & Business Impact

Exploiting this vulnerability chain grants threat actors immediate and permanent control over targeted user accounts. While session token theft typically yields temporary access that expires alongside the cookie, combining DOM XSS and missing HttpOnly protections with an unthrottled re-authentication endpoint enables attackers to permanently overwrite victim credentials. Because the application lacks an out-of-band account recovery mechanism (such as email-based password resets), victims are left with no self-service path to reclaim their access. This leads to total victim lockout, unauthorized access to sensitive personal data, and potential business reputation damage.

Remediation Guidance

1. Replace Insecure DOM Sinks with Safe Text Assignment: Refactor frontend search component code to assign user input to safe text properties (textContent or innerText) rather than innerHTML. Alternatively, apply contextual HTML entity encoding to query parameters prior to DOM insertion.

2. Deploy a Restrictive Content Security Policy (CSP): Configure a Content Security Policy header using script-src 'self' and img-src 'self'. Explicitly omitting 'unsafe-inline' from script-src ensures the browser blocks all inline JavaScript execution—preventing script execution both within <script> tags and via inline event attributes (onerror, onload) on elements like <img>.

3. Enforce HttpOnly Attribute on Session Cookies: Configure session management middleware to append the HttpOnly security flag to all authentication cookies (Set-Cookie: token=...; Secure; HttpOnly; SameSite=Lax). This prevents client-side JavaScript execution from accessing sensitive token objects via document.cookie.

4. Apply Rate Controls Across Authenticated State-Changing Routes: Extend sliding-window rate limiting to /rest/user/change-password (e.g., max 3–5 failed attempts per 15 minutes per IP/session). Establishing explicit rate controls independently of authentication state ensures re-authentication handlers remain protected against automated credential fuzzing during post-exploitation scenarios.

5. Eliminate Response Oracles via Uniform HTTP Redirection: Configure the backend handler to issue a uniform HTTP GET redirect back to the change password page upon form submission regardless of success or failure. This automatically eliminates HTTP status code and response length differentials on the wire. Specific error notifications should be loaded and dynamically populated into the client-side DOM via frontend scripts, neutralizing automated response-length analysis (such as Burp Intruder).

Finding 2.3: Insecure State-Changing Operations on Profile Management Endpoints via Cross-Site Request Forgery (CSRF)

SEVERITY: MEDIUM	CWE-ID: CWE-352	CVSS SCORE: 5.4
AFFECTED POINT	POST /profile/image/url (imageUrl parameter) & POST /profile (username parameter)	
OWASP TOP 10	A01:2025 – Broken Access Control	

Vulnerability Description & Technical Mechanics

The profile management endpoints (POST /profile/image/url and POST /profile) fail to validate anti-CSRF tokens, SameSite cookie restrictions (SameSite=Lax or Strict), or custom request headers (Authorization: barer <JWT>). This exposes the user profile subsystem to Cross-Site Request Forgery (CSRF) across two state-changing vectors:

• Vector 1 — Profile Picture/Avatar Defacement (POST /profile/image/url): The avatar image URL handler accepts cross-site POST requests without anti-CSRF validation. An attacker can host a cross-site HTML form containing a pre-selected image URL parameter (imageUrl) pointing to an attacker-hosted asset (http://attacker-site.local:8000/cat.png) and automatically trigger an update when an authenticated victim visits the page. Because avatars render publicly across reviews and community interactions, forcing an arbitrary image update introduces immediate profile defacement, social engineering, and brand reputational risk.

• Vector 2 — Cross-Site Username Modification (POST /profile): The profile endpoint processes username updates (username parameter) without requiring custom Authorization headers or anti-CSRF tokens. Because Mozilla Firefox leaves unflagged session cookies unconstrained by default (treating them as SameSite=None), cross-site POST requests automatically include victim session credentials. Conversely, Chromium browsers enforce SameSite=Lax defaults, restricting exploitation on Chrome to Chromium's 2-minute "Lax-allowing-unsafe" window post-authentication.

Steps to Reproduce (Proof of Concept)

Vector 1: Profile Avatar Defacement

1. Intercept POST /profile/image/url in Burp Suite and observe that the endpoint handles requests without verifying anti-CSRF tokens or custom headers.

2. Host an auto-submitting cross-site form/exploit snippet on an attacker-controlled origin (http://attacker-site.local:8000) targeting POST /profile/image/url:

```
<form id="csrfForm" action="http://localhost:3000/profile/image/url" method="POST">
  <input type="hidden" name="imageUrl" value="http://attacker-site.local:8000/cat.png" />
</form>
<script>
  document.getElementById('csrfForm').submit();
</script>
```

3. Lure the authenticated victim to visit the exploit link in Firefox (or within Chromium's 2-minute post-login window). The victim's browser automatically submits the payload alongside session cookies, resulting in an HTTP 200 OK response.

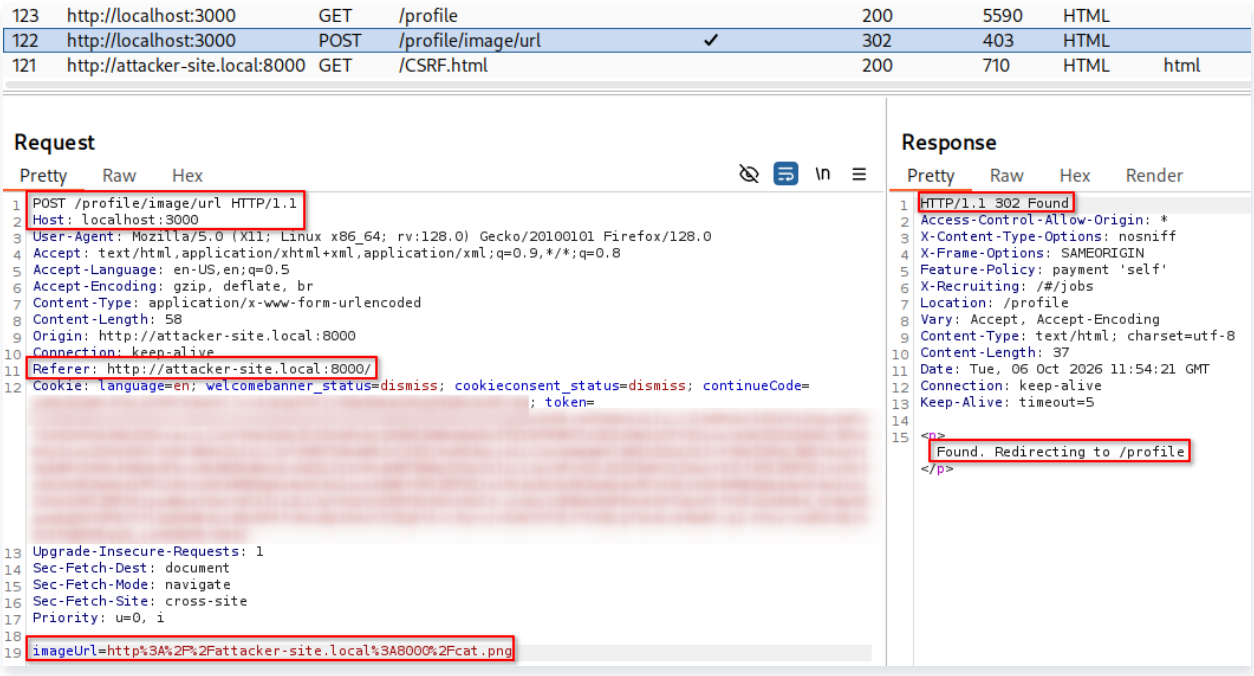


Figure 2.3A: Burp Suite Repeater capture of the cross-origin POST request targeting /profile/image/url, demonstrating backend acceptance (HTTP 200 OK) of the forced imageUrl parameter without anti-CSRF token verification or preflight validation

4. Inspect the application user profile interface under the victim's active session to verify that the avatar asset has been overwritten and successfully rendered across the web interface.



Figure 2.3B: Application user profile interface rendered under the victim's active session, confirming the successful cross-site avatar overwrite with the attacker-hosted payload asset (cat.png)

Vector 2: Cross-Site Username Modification

5. Authenticate as a valid victim, navigate to profile settings, and capture POST /profile in Burp Suite. Confirm the endpoint processes requests without custom headers or anti-CSRF tokens.

6. Host an auto-submitting cross-site exploit snippet on an attacker-controlled origin (http://attacker-site.local:8000) targeting POST /profile:

```
<form id="csrfForm" action="http://localhost:3000/profile" method="POST">
  <input type="hidden" name="username" value="csrf_proof" />
</form>
<script>
  document.getElementById('csrfForm').submit();
</script>
```

7. Lure the authenticated victim to visit the exploit link in Firefox (or within Chromium's 2-minute post-login window). The browser attaches session cookies, the server returns HTTP 200 OK, and the victim's username is updated to csrf_proof.



Figure 2.3C: Cross-site POST request execution in Mozilla Firefox targeting /profile, demonstrating automated account parameter modification to "csrf_proof" due to unconstrained default session cookie handling and missing anti-CSRF protections

Risk & Business Impact

While unauthorized username modification is a low-severity nuisance, combining it with cross-site avatar replacement elevates the finding to MEDIUM severity. Forcing arbitrary or offensive imagery onto a victim's public profile introduces severe defacement, impersonation, and social engineering risks across public reviews and comments. Furthermore, because Firefox leaves unflagged session cookies unconstrained (SameSite=None default behavior), users on Firefox remain persistently exposed to silent account tampering, whereas Chromium users are vulnerable during the 2-minute post-login window.

Remediation Guidance

1. **Centralize Anti-CSRF Protection via Router Middleware:** Configure router middleware to verify cryptographically secure anti-CSRF tokens or custom request headers (Authorization: Bearer <JWT>) on all state-changing HTTP methods (POST, PUT, DELETE, PATCH).
2. **Enforce Explicit SameSite=Strict Cookie Attributes:** Set SameSite=Strict; Secure; HttpOnly on all session cookies to prevent browsers from attaching session credentials to cross-site requests.
3. **Disallow Simple Cross-Origin Requests on Upload Routes:** Require endpoints to accept only non-simple content-types (such as application/json) or custom headers (like X-Requested-With). Standard HTML forms cannot set these attributes, forcing browsers to issue a CORS preflight (OPTIONS) check that blocks cross-origin CSRF attacks automatically.

Finding 2.4: Primary Login Vulnerability to unrestricted Credential Stuffing

SEVERITY: CRITICAL	CWE-ID: CWE-307	CVSS SCORE: 8.1
AFFECTED POINT	POST /rest/user/login (password JSON parameter)	
OWASP TOP 10	A07:2025 – Identification and Authentication Failures	

Vulnerability Description & Technical Mechanics

The primary login endpoint (POST /rest/user/login) accepts unlimited sequential authentication attempts without enforcing IP-based rate limiting, account lockout policies, or CAPTCHA challenges. Attackers can launch automated dictionary and credential stuffing attacks against the password JSON body parameter. Furthermore, observable response differentials (HTTP 401 length 413 vs HTTP 200 length 1191) allow automated scripts to confirm valid credentials instantly without server-side interference.

Steps to Reproduce (Proof of Concept)

1. Configure Automated Payload Delivery: Intercept a valid login request targeting POST /rest/user/login and send it to Burp Intruder. Define the payload position on the "password" JSON field and load a credential dictionary.

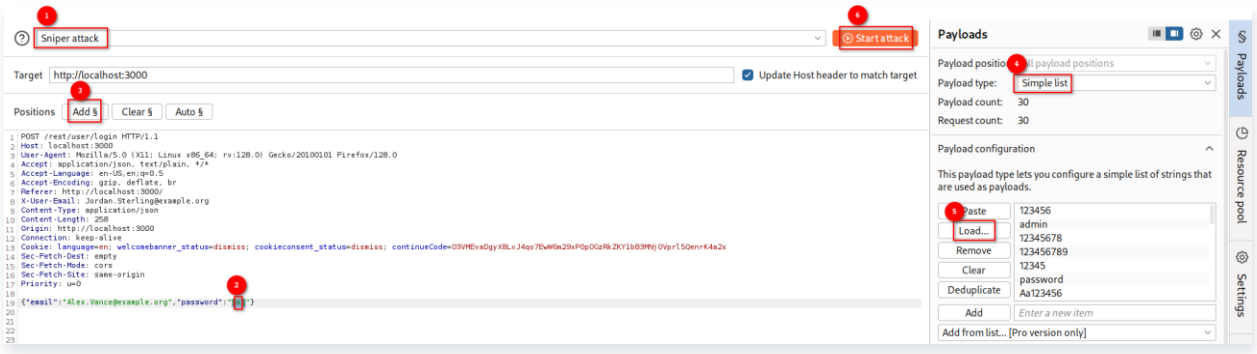


Figure 2.4A: Burp Intruder configuration targeting the password JSON field

2. Execute Attack: Execute the attack. Observe that 110 failed attempts yield HTTP 401 Unauthorized (413 bytes), whereas the correct credential (Password) returns HTTP 200 OK (1191 bytes), allowing automated scripts to identify valid credentials instantly.

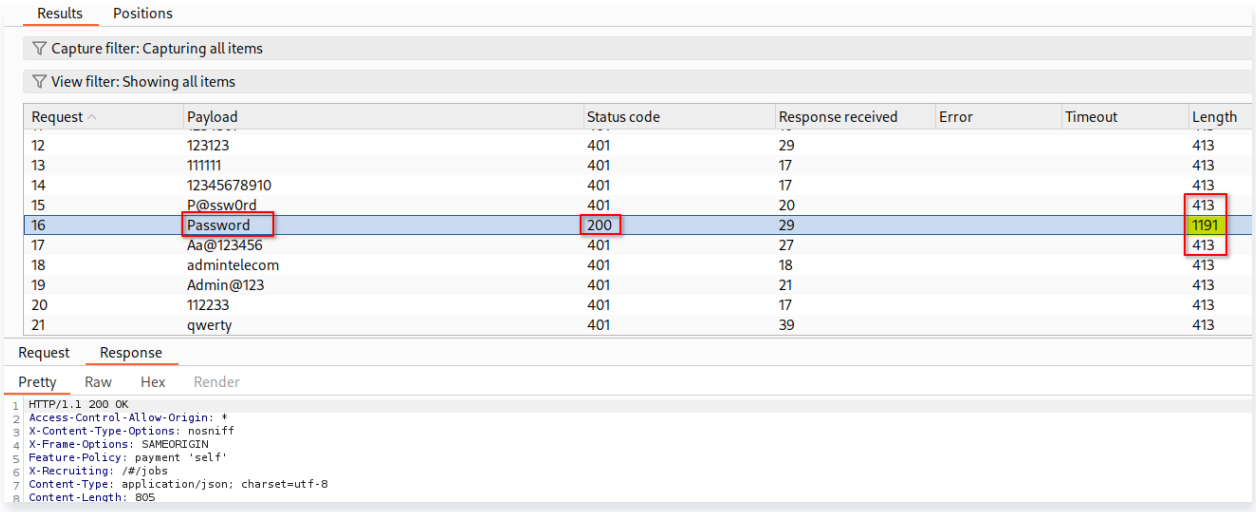


Figure 2.4B: Intruder attack results displaying successful login response identified via status codes or payload length anomaly

Risk & Business Impact

Enables high-volume credential stuffing and dictionary attacks against all platform accounts. Attackers can compromise user accounts, access sensitive personal data, and perform unauthorized account actions without triggering server-side defenses.

Remediation Guidance

1. Implement Adaptive Rate Limiting & Anti-Automation Controls (Core Patch): Restrict authentication attempts on POST /rest/user/login to a maximum of 5 failed attempts per 5-minute window.

MANDATORY TECHNICAL BLUEPRINT:
• Atomic Memory Store Operations: Execute counter updates using atomic memory store commands (e.g., Redis INCR with EXPIRE or atomic Lua scripts).
• CAPTCHA Escalation: Deploy a mandatory CAPTCHA challenge (e.g., reCAPTCHA v3 or Cloudflare Turnstile) after 3 consecutive failed attempts on an account.

2. Consequences of Naive Implementation: Relying on non-atomic application-level logic introduces concurrency race conditions, allowing thousands of parallel requests sent within milliseconds to be processed as if they were a single request—completely bypassing any naively implemented 5-attempt rate limit. Furthermore, failing to deploy CAPTCHA leaves the endpoint exposed to high-speed automated fuzzing and dictionary attacks via tools like Burp Intruder.

SECTION 3: VERIFIED DEFENSIVE CONTROLS (PASS OBSERVATIONS)

As part of our holistic testing methodology, offensive security engagements evaluate not only exploitable vulnerabilities but also active defensive mechanisms that successfully mitigate potential attack vectors. The controls below were explicitly tested and confirmed to be effectively implemented by the target application.

CTRL-01: Input Sanitization & Output Encoding on User Profile Subsystem

Evaluated Target: POST /profile (Username Update)

Test Methodology: Injected executable tags (<script>), custom elements (<testpayload>), empty brackets (<>), structural HTML (<a>,), and attribute breakouts (value="user_input") into the username parameter.

Observed Behavior: Input is handled safely across four distinct mechanics:
• Executable/Custom Tags (<script>, <testpayload>): Completely stripped server-side.
• Empty Brackets (<>): Rendered literally as-is.
• Structural HTML (<a>,): Any opening or closing tag is automatically balanced via parser auto-correction and DOM repair without executing embedded code.
• Attribute Context (value="user_input"): Double quotes (") are HTML-entity encoded (to "), preventing quote breakout.

CTRL-02: Array Input Rejection & Parameter Pollution Resistance on Login Payload

Evaluated Target: POST /rest/user/login

Test Methodology: Submitted a JSON array in the password field ("password":["pass_guess_1", "pass_guess_2", ...]) to test for array-based credential batching and parameter pollution.

Observed Behavior: The server successfully rejected array iteration, confirming resistance to batch-login array abuse and preventing attackers from evaluating multiple password guesses within a single request context.

CTRL-03: Response Timing Uniformity & Side-Channel Resistance on Login

Evaluated Target: POST /rest/user/login

Test Methodology: Measured server response latency across varying payload lengths (1-character vs 1,000+ characters) against known valid (Alex.Vance@example.org) versus guaranteed non-existent (invalid_user@example.org) user accounts.

Observed Behavior: Server response times were uniform (~1,041 ms for valid vs ~1,056 ms for invalid). The consistent ~1,000 ms baseline confirms the use of an intentionally time-costly password hashing algorithm (e.g., bcrypt/argon2) and constant-time string comparison, mitigating username enumeration and timing side-channels.

CTRL-04: Generic Error Messaging & Response Uniformity on Primary Authentication

Evaluated Target: POST /rest/user/login

Test Methodology: Submitted invalid passwords against a known valid account (Alex.Vance@example.org) versus a guaranteed non-existent account (nonexistent_user_99@example.org) and analyzed HTTP status codes, byte lengths, and hex variance in Burp Comparer.

Observed Behavior: The server returned identical 401 Unauthorized status codes, identical response body byte lengths (413 bytes), and the exact same generic error message ("Invalid email or password.") across both transactions. Response variance was restricted strictly to dynamic server headers (Date: timestamp shift), successfully preventing account enumeration.

CTRL-05: Synchronous Atomic Processing & Parallel Rate-Limiting Resistance on Account Recovery

Evaluated Target: POST /rest/user/reset-password

Test Methodology: Issued a batch of 10 concurrent parallel requests targeting the answer JSON field (9 invalid guesses, 1 valid guess) with an active rate-limit counter sitting at 99 remaining attempts, utilizing HTTP/1.1 last-byte connection synchronization.

Observed Behavior: The server processed each request synchronously and atomically, decrementing the counter individually from 99 down to 90. The server rejected HTTP/2 negotiation (400 Bad Request) and strictly enforced atomic request sequencing without race condition vulnerability under parallel load.

CTRL-06: Pitchfork Attack Counter Persistence on Account Recovery

Evaluated Target: POST /rest/user/reset-password

Test Methodology: Executed a dual-parameter Pitchfork attack in Burp Intruder targeting the email and security question answer JSON fields, interleaving valid requests at regular intervals (every 4 requests) to evaluate whether valid requests reset or bypass the attempt/rate-limit counter.

Observed Behavior: The active rate-limit counter persisted continuously across valid and invalid requests without resetting, confirming that interleaving valid account requests does not clear or tamper with rate-limiting enforcement state.

CTRL-07: HTTP Method Override Directives Rejection on State Changing Routes

Evaluated Target: POST /profile & POST /rest/user/change-username

Test Methodology: Issued cross-origin POST requests to evaluate state modification bypasses via GET navigations, systematically testing HTTP method override techniques across URL query strings and request body parameters (method, _method), as well as custom headers (X-HTTP-Method-Override, X-HTTP-Method, X-Method-Override).

Observed Behavior: The backend router strictly rejected cross-site state modification attempts when method override vectors were used—whether supplied as URL query parameters, request body fields, or custom HTTP headers—returning HTTP 500 errors and successfully preventing method tunneling.

SECTION 4: APPENDICES & FORMAL SIGN-OFF

Appendix A: CVSS v3.1 Severity Rating Framework

Vulnerability severity ratings in this report are assigned using the FIRST Common Vulnerability Scoring System (CVSS v3.1) standard, mapped directly to OWASP Top 10:2025 categories to provide transparent risk evaluation.

SEVERITY	TARGET REMEDIATION SLA	CVSS v3.1 RANGE	DESCRIPTION & IMPACT
CRITICAL (P1)	**Immediate (24–48 Hours)**	9.0–10.0	Flaws permitting immediate, unauthenticated system compromise, full account takeover, or arbitrary code execution.
HIGH (P2)	**14 Days**	7.0–8.9	High-impact flaws enabling significant data exposure, privilege escalation, or full session hijacking.
MEDIUM (P3)	**30 Days**	4.0–6.9	Flaws requiring specific user interaction or specific environmental conditions to impact confidentiality, integrity, or availability.
LOW / INFO (P4)	**60 Days**	0.1–3.9	Minor security defects, configuration weaknesses, or defense-in-depth gaps with limited impact or high exploitation complexity.
INFORMATIONAL (P5)	**90 Days**	0.0	Non-vulnerable security best-practice recommendations, architectural hardening suggestions, or operational hygiene items.

Qualitative Risk Escalation Note: *While CVSS v3.1 base metrics quantify mathematical vulnerability characteristics, qualitative severity ratings in this report may be elevated to CRITICAL based on contextual business impact—specifically chained vulnerability paths that result in unmitigated Account Takeover (ATO).*

Appendix B: Developer Remediation Action Plan Checklist

This checklist organizes technical mitigations by priority to guide engineering teams in systematically remediating identified risks:

PRIORITY	REMEDIATION ACTION ITEM
P1 - CRITICAL	**[FIND-02]** Refactor DOM Sinks to textContent / innerText & HTML Entity Encoding; Deploy Strict Content Security Policy (CSP) Directives (script-src 'self'); Enforce Secure HttpOnly Cookie Flags; Enforce Rate Limiting & Re-Authentication on Password Changes
P1 - CRITICAL	**[FIND-01]** Configure Server-Side Uniform Redirection & Generic Feedback; Disable Automatic Security Question Prompt Pre-Filling; Enforce Sliding-Window Rate Limiting on Password Recovery Handler
P1 - CRITICAL	**[FIND-04]** Enforce Adaptive Sliding-Window Rate Limiting & Account Lockout; Deploy Progressive CAPTCHA Challenges on Repeated Login Failures; Implement Global Exception Handling & Strict Data Type Validation
P3 - MEDIUM	**[FIND-03]** Centralize Anti-CSRF Token Validation Middleware; Enforce Explicit SameSite=Strict Cookie Attributes; Disallow Simple Cross-Origin Requests on Upload Routes

Appendix C: Assessment Environment & Testing Tooling

Testing was performed in a local staging environment deploying OWASP Juice Shop (v20.2.0) using **Burp Suite Community Edition (v2025.3.4)** for live HTTP request interception, parameter tampering, manual payload execution via Repeater parallel groups (utilizing last-byte sync) and Intruder (routed via **FoxyProxy**), and HTTP response header analysis in Repeater to detect unflagged Set-Cookie directives missing explicit SameSite attributes (evaluating cross-browser SameSite=Lax defaults and Chromium's 120-second post-login window). Client-side security auditing relied on **Chromium DevTools** integrated with PortSwigger **DOM Invader** to trace location.search source data flows into innerHTML sinks. All test vectors—including security question prompt probing, response byte-length oracle profiling, unthrottled password re-authentication brute-forcing, and JSON array parameter injection—were manually engineered using non-harmful custom security payloads.

Appendix D: Artifact Cleanup Attestation & Formal Sign-Off

Artifact Cleanup Attestation: The Principal Security Assessor formally attests that all temporary test accounts, fuzzed security question responses, test payload strings, session tokens, and modified profile parameters generated during the active testing window (September 16–25, 2026) were safely purged from the target staging deployment upon engagement completion.

Legal Disclaimer & Confidentiality Notice

This document is a technical security evaluation report produced by Verge Security Labs solely for demonstration and portfolio evaluation purposes. Technical findings reflect security evaluations conducted against a controlled, open-source target environment (OWASP Juice Shop v20.2.0). To maintain realistic technical fidelity, HTTP request schemas, host paths, and target endpoints are preserved in complete operational detail, while authentication tokens, session headers, and user credentials have been scrubbed and sanitized. This deliverable contains no proprietary corporate assets. All text, layout architecture, and report intellectual property herein remain the exclusive property of Verge Security Labs. Unauthorized copying, re-branding, plagiarized redistribution, or commercial reproduction of this report without explicit written consent is strictly prohibited.

Lead Assessor Formal Sign-Off

NAME / ENTITY	TITLE / ROLE	ORGANIZATION	STATUS
Alex Vance	Principal Security Assessor	Verge Security Labs	DIGITALLY SIGNED & VERIFIED